source removal

```
#include <stdio.h>

int main() {
   int n, u, v, adj[100][100] = {0}, indeg[100] = {0};
   int q[100], front = 0, rear = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter edges (-1 -1 to quit):\n");
   while (scanf("%d %d", &u, &v) == 2 && u != -1 && v != -1) {
      if (u < 0 || v < 0 || u >= n || v >= n) { printf("Invalid edge!\n"); continue; }
      if (!adj[u][v]) { adj[u][v] = 1; indeg[v]++; }
   }

   for (v = 0; v < n; v++)               /* sources: indegree 0 */
      if (!indeg[v]) q[rear++] = v;

   while (front < rear) {                /* remove a source, delete its edges */
      u = q[front++];
      for (v = 0; v < n; v++)
         if (adj[u][v] && --indeg[v] == 0)
            q[rear++] = v;
   }

   if (rear < n)
      printf("No topological ordering possible, graph contains a cycle\n");
   else {
      printf("Vertices in topological order are:\n");
      for (v = 0; v < n; v++) printf("%d ", q[v]);
      printf("\n");
   }
   return 0;
}
```

n-queens

```
#include <stdio.h>
#include <stdlib.h>

int a[30];
int count = 0;

/* Is the queen in row pos safe from the queens in rows 1..pos-1? */
int place(int pos)
{
   int i;
   for (i = 1; i < pos; i++) {
      if (a[i] == a[pos] || abs(a[i] - a[pos]) == abs(i - pos))
         return 0;
   }
   return 1;
}

/* Print the current solution as a grid of Q and * */
void print_sol(int n)
{
   int i, j;
   count++;
   printf("\n\nSolution #%d:\n", count);
   for (i = 1; i <= n; i++) {
      for (j = 1; j <= n; j++) {
         if (a[i] == j)
            printf("Q\t");
         else
            printf("*\t");
      }
      printf("\n");
   }
}

/* Backtracking search for all solutions */
void queen(int n)
{
   int k = 1;
   a[k] = 0;
   while (k != 0) {
      a[k] = a[k] + 1;
      while (a[k] <= n && !place(k))
         a[k]++;
      if (a[k] <= n) {
         if (k == n)
            print_sol(n);
         else {
            k++;
            a[k] = 0;
         }
      }
      else
         k--;
   }
}

int main()
{
   int n;
   printf("Enter the number of Queens\n");
   scanf("%d", &n);
   queen(n);
   printf("\nTotal solutions=%d\n", count);
   return 0;
}
```

dfs top

```
#include <stdio.h>

int main()
{
   int a[10][10], indegree[10] = {0}, s[10], top = -1, n, i, j, u;

   printf("Enter number of nodes:");
   scanf("%d", &n);

   printf("\nEnter the adjacency matrix:\n");
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++)
      {
         scanf("%d", &a[i][j]);
         indegree[j] += a[i][j]; //
      }

   for (i = 0; i < n; i++)
      if (indegree[i] == 0)
         s[++top] = i;

   printf("The topological Sequence is:\n");
   while (top != -1)
   {
      u = s[top--];
      printf("%d ", u);
      for (j = 0; j < n; j++)
         if (a[u][j] == 1 && --indegree[j] == 0)
            s[++top] = j;
   }
   return 0;
}
```

matrix-multi

```
#include <stdio.h>
#include <limits.h>

int MatrixChainOrder(int p[], int n)       // <-- covered (inputs)
{
   int m[n][n];                        // <-- NEXT: the memory table
   int i, j, k, L, q;

   for (i = 1; i < n; i++)
      m[i][i] = 0;           mi^2 = 0         // later

   for (L = 2; L < n; L++)               // later
   {
      for (i = 1; i <= n-L+1; i++)
      {
         j = i+L-1;
         m[i][j] = INT_MAX;
         for (k = i; k <= j-1; k++)
         {
            q = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];  // formula covered
            if (q < m[i][j])
               m[i][j] = q;
         }
      }
   }
   return m[1][n-1];
}

int main()
{
   int arr[] = {30, 35, 15, 5, 10, 20, 25};   // <-- covered
   int size = sizeof(arr)/sizeof(arr[0]);     // <-- covered
   printf("Minimum number of multiplications is:\n");
   printf("%d\n", MatrixChainOrder(arr, size));
   return 0;
}
```

horspool

```
#include <stdio.h>
#include <string.h>

int main() {
   char t[100], p[25];
   int table[256];
   int n, m, i, k, flag = 0;

   printf("Enter the text : ");
   scanf("%s", t);
   n = strlen(t);

   printf("Enter the pattern : ");
   scanf("%s", p);
   m = strlen(p);

   /* Phase 1: build the shift table */
   for (i = 0; i < 256; i++)
      table[i] = m;                    /* default: jump m */
   for (i = 0; i < m - 1; i++)
      table[(unsigned char)p[i]] = m - 1 - i;   monei

   /* Phase 2: search */
   i = m - 1;                          /* text index under pattern's last letter */
   while (i <= n - 1) {
      k = 0;                           /* compare right to left */
      while (k <= m - 1 && p[m - 1 - k] == t[i - k])
         k++;

      if (k == m) {                    /* all m letters matched */
         printf("The position of the pattern is %d\n", i - m + 2);
         flag = 1;
         break;
      }
      else
         i = i + table[(unsigned char)t[i]];   /* mismatch: jump */
   }

   if (!flag)
      printf("Pattern is not found in the given text\n");

   return 0;
}
```

dijsktra

```
#include <stdio.h>
#define INF 999

int main() {
   int n, u, v, i, j, w, min, count;
   int cost[20][20], dist[20], flag[20];

   printf("Enter number of nodes: ");
   scanf("%d", &n);
   printf("Enter cost matrix (0 = no edge):\n");
   for (i = 1; i <= n; i++)
      for (j = 1; j <= n; j++) {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0) cost[i][j] = INF;
      }

   printf("Enter source node: ");
   scanf("%d", &u);              /* u = source */

   for (i = 1; i <= n; i++) {
      flag[i] = 0;
      dist[i] = cost[u][i];
   }
   dist[u] = 0;
   flag[u] = 1;

   for (count = 2; count <= n; count++) {
      min = INF;
      for (w = 1; w <= n; w++)
         if (!flag[w] && dist[w] < min) {
            min = dist[w];
            v = w;            /* v = closest unsettled node */
         }
      flag[v] = 1;
      for (w = 1; w <= n; w++)
         if (!flag[w] && dist[v] + cost[v][w] < dist[w])
            dist[w] = dist[v] + cost[v][w];
   }

   printf("Shortest paths:\n");
   for (i = 1; i <= n; i++)
      if (i != u)
         printf("%d->%d, cost=%d\n", u, i, dist[i]);
   return 0;
}
```

prims

```
#include<stdio.h>

int a,b,n,i,j,ne=1;
int visited[10]={0},min,mincost=0,cost[10][10];

int main()
{
   printf("\n Enter the number of nodes:");
   scanf("%d",&n);

   printf("\n Enter the adjacency matrix:\n");
   for(i=1;i<=n;i++)
      for(j=1;j<=n;j++)
      {
         scanf("%d",&cost[i][j]);
         if(cost[i][j]==0)
            cost[i][j]=999;
      }

   visited[1]=1;
   printf("\n");

   while(ne<n)
   {
      for(i=1,min=999;i<=n;i++)
         for(j=1;j<=n;j++)
            if(cost[i][j]<min)
               if(visited[i]!=0)
               {
                  min=cost[i][j];
                  a=i;
                  b=j;
               }

      if(visited[a]==0 || visited[b]==0)
      {
         printf("\n Edge %d:(%d %d) cost:%d",ne++,a,b,min);
         mincost+=min;
         visited[b]=1;
      }

      cost[a][b]=cost[b][a]=999;
   }

   printf("\n Minimum cost=%d",mincost);
   return 0;
}
```

quick sort

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *p, int *q) {
   int t = *p;
   *p = *q;
   *q = t;
}

void quickSort(int a[], int low, int high) {
   if (low >= high) return;

   int key = a[low];
   int i = low + 1;
   int j = high;

   while (i <= j) {
      while (i <= high && a[i] <= key)   // move i right
         i = i + 1;
      while (a[j] > key)               // move j left
         j = j - 1;
      if (i < j)
         swap(&a[i], &a[j]);
   }
   swap(&a[low], &a[j]);               // put key in its place

   quickSort(a, low, j - 1);
   quickSort(a, j + 1, high);
}

int main() {
   int n, a[10000];

   printf("Enter number of elements: ");
   scanf("%d", &n);

   for (int k = 0; k < n; k++) {
      a[k] = rand();
      printf("%d\t", a[k]);
   }

   clock_t st = clock();
   quickSort(a, 0, n - 1);
   clock_t et = clock();

   printf("\nSorted numbers:\n");
   for (int k = 0; k < n; k++)
      printf("%d\t", a[k]);

   printf("\nTime taken: %f seconds\n", (double)(et - st) / CLOCKS_PER_SEC);
   return 0;
}
```

dfs

```
#include <stdio.h>

int graph[20][20], visited[20], n;

void dfs(int node)
{
   int neighbour;
   if (visited[node])
      return;
   visited[node] = 1; // mark that node as visited.
   for (neighbour = 1; neighbour <= n; neighbour++)
      if (graph[node][neighbour] && !visited[neighbour])
      {
         printf("\n %d->%d", node, neighbour);
         dfs(neighbour);
      }
}

int main()
{
   int i, j, count = 0;

   printf("\n Enter number of vertices:");
   scanf("%d", &n);

   printf("\n Enter the adjacency matrix:\n");
   for (i = 1; i <= n; i++)
      for (j = 1; j <= n; j++)
         scanf("%d", &graph[i][j]);

   dfs(1);
   printf("\n");

   for (i = 1; i <= n; i++)
      if (visited[i])
         count++;

   if (count == n)
      printf("\n Graph is connected");
   else
      printf("\n Graph is not connected");

   return 0;
}
```

longest common sub

```
#include <iostream>
#include <string>
using namespace std;

int solve(int i, int j, string &text1, string &text2) {
   if (i == text1.length() || j == text2.length()) {
      return 0;
   }

   if (text1[i] == text2[j]) {
      return 1 + solve(i + 1, j + 1, text1, text2);
   }

   return max(
      solve(i + 1, j, text1, text2),
      solve(i, j + 1, text1, text2)
   );
}

int longestCommonSubsequence(string text1, string text2) {
   return solve(0, 0, text1, text2);
}

int main() {
   string text1 = "abcbdab";
   string text2 = "bdcaba";

   cout << endl << "Length of X[] is = " << text1.length() << endl;
   cout << endl << "Length of Y[] is = " << text2.length() << endl;
   cout << endl << "Length of LCS is = " << longestCommonSubsequence(text1, text2) << endl;
   return 0;
}
```

kruskal’s

```
#include <stdio.h>
#include <stdlib.h>

int i, j, k, a, b, n, ne = 1;
int min, mincost = 0, cost[9][9], parent[9];

int find(int);
int uni(int, int);

int main() {
   printf("\nImplementation of Kruskal's algorithm\n");
   printf("\nEnter the no. of vertices\n");
   scanf("%d", &n);

   printf("\nEnter the cost adjacency matrix\n");
   for (i = 1; i <= n; i++) {
      for (j = 1; j <= n; j++) {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0)
            cost[i][j] = 999;
      }
   }

   printf("\nThe edges of Minimum Cost Spanning Tree are\n\n");
   while (ne < n) {
      for (i = 1, min = 999; i <= n; i++) {
         for (j = 1; j <= n; j++) {
            if (cost[i][j] < min) {
               min = cost[i][j];
               a = i;
               b = j;
            }
         }
      }
      u = find(u);
      v = find(v);
      if (uni(u, v)) {
         printf("\n%d edge (%d,%d) = %d\n", ne++, a, b, min);
         mincost += min;
      }
      cost[a][b] = cost[b][a] = 999;
   }
   printf("\n\tMinimum cost = %d\n", mincost);
   return 0;
}

int find(int i) {
   while (parent[i])
      i = parent[i];
   return i;
}

int uni(int i, int j) {
   if (i != j) {
      parent[j] = i;
      return 1;
   }
   return 0;
}
```

merge

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int B[10000];   // extra array used by Merge()

void Merge(int A[], int low, int mid, int high) {
   int i = low, j = mid + 1, k = low;

   while (i <= mid && j <= high) {
      if (A[i] <= A[j]) {
         B[k] = A[i];
         i++;
         k++;
      }
      else {                  // A[i] > A[j]
         B[k] = A[j];
         j++;
         k++;
      }
   }

   if (i > mid) {               // left half finished, copy rest of right half
      while (j <= high) {
         B[k] = A[j];
         j++;
         k++;
      }
   }
   else {                      // j > high: right half finished, copy rest of left half
      while (i <= mid) {
         B[k] = A[i];
         i++;
         k++;
      }
   }

   for (k = low; k <= high; k++)   // copy sorted part from B back to A
      A[k] = B[k];
}

void MergeSort(int A[], int low, int high) {
   if (low < high) {
      int mid = (low + high) / 2;
      MergeSort(A, low, mid);
      MergeSort(A, mid + 1, high);
      Merge(A, low, mid, high);
   }
}

int main() {
   int n, a[10000];
   printf("Enter number of elements: ");
   scanf("%d", &n);

   printf("\nThe random numbers are:\n");
   for (int k = 0; k < n; k++) {
      a[k] = rand();
      printf("%d\t", a[k]);
   }

   clock_t st = clock();
   MergeSort(a, 0, n - 1);
   clock_t et = clock();

   printf("\nSorted numbers:\n");
   for (int k = 0; k < n; k++)
      printf("%d\t", a[k]);

   printf("\nTime taken: %e seconds\n", (double)(et - st) / CLOCKS_PER_SEC);
   return 0;
}
```

comb sum

```
#include <stdio.h>

int w[50], curr[50], n, t, found;

void solve(int idx, int target, int size) {
   if (target < 0) return;              // picked too much: dead end
   if (target == 0) {                   // exact sum: print the subset
      printf("\n{");
      for (int j = 0; j < size; j++) printf("%3d", curr[j]);
      printf(" }\n");
      found = 1;
      return;
   }
   for (int i = idx; i < n; i++) {
      curr[size] = w[i];                // DO: pick w[i]
      solve(i + 1, target - w[i], size + 1); // EXPLORE
                                        // UNDO: next iteration overwrites curr[size]
   }
}

int main() {
   printf("Enter how many numbers: ");
   scanf("%d", &n);
   printf("Enter %d numbers: ", n);
   for (int i = 0; i < n; i++) scanf("%d", &w[i]);
   printf("Enter the sum value: ");
   scanf("%d", &t);                     // t = the user's target sum

   for (int i = 0; i < n; i++)          // bubble sort, ascending
      for (int j = 0; j < n - 1 - i; j++)
         if (w[j] > w[j+1]) { int tmp = w[j]; w[j] = w[j+1]; w[j+1] = tmp; }

   solve(0, t, 0);                      // start: target = t, nothing picked
   if (!found) printf("\nSubset construction is not possible");
   return 0;
}
```

bfs

```
#include <stdio.h>

int graph[20][20], visited[20], queue[20], n, front = 1, rear = 0;

void bfs(int node)
{
    int neighbour;
    queue[++rear] = node;
    visited[node] = 1; // mark that node as visited.
    while (front <= rear)
    {
        node = queue[front++];
        for (neighbour = 1; neighbour <= n; neighbour++)
            if (graph[node][neighbour] && !visited[neighbour])
            {
                visited[neighbour] = 1;
                queue[++rear] = neighbour;
            }
    }
}

int main()
{
    int i, j, u;

    printf("\n Enter number of vertices:");
    scanf("%d", &n);

    printf("\n Enter the adjacency matrix:\n");
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
            scanf("%d", &graph[i][j]);

    printf("\n Enter the starting vertex:");
    scanf("%d", &u);

    bfs(u);

    for (i = 1; i <= n; i++)
        if (visited[i])
            printf("%d\t", i);

    return 0;
}
```